# Computer Vision DermaCheck: clasificación de lesiones cutáneas con MobileNetV2 en una Raspberry Pi 5

Salvador Martinez[1*], Christopher Yañez[1*], Mauricio Chavez[1†] and Miguel Valdez[1†]

[1*]PrepaTec Campus Cuernavaca, Tecnológico de Monterrey, Autopista del Sol km 104+060, Col. Real del Puente, Xochitepec, 62790, Morelos, México.

*Autores de correspondencia. Correos: a01425829@tec.mx; a01426168@tec.mx;
Coautores: a01426058@tec.mx; a01739293@tec.mx;
†Estos autores contribuyeron por igual a este trabajo.

**Resumen**

Un melanoma y un lunar común se parecen lo bastante como para despistar a simple vista incluso a personal entrenado, y el dermatoscopio, que ayuda a separarlos, rara vez sale de un consultorio. Armamos Computer Vision DermaCheck, un prototipo de bajo costo que clasifica fotografías de lesiones cutáneas en las ocho categorías del conjunto ISIC 2019. Partimos de una red MobileNetV2 preentrenada con ImageNet y la ajustamos en dos fases con 18,237 imágenes dermatoscópicas. Como unas clases tienen muchísimas más imágenes que otras, ponderamos la pérdida para compensar ese desbalance. Todo el entrenamiento corrió en el CPU de una laptop y duró 44.6 minutos. Ya exportado a TensorFlow Lite, el modelo trabaja en una Raspberry Pi 5 dentro de una aplicación táctil que enseña la imagen en vivo con un cuadro guía y, tras la captura, las tres clases más probables. Frente a 5,067 imágenes que la red nunca había visto logró una exactitud de 0.725 y un recall promedio por clase de 0.632. Atina con los nevos (0.855) y con las lesiones vasculares (0.862). Pero de los melanomas reconoce solo el 47.1 %. En la Pi, cada imagen tarda 9.7 ms con cuatro hilos, y con el modelo completo sus predicciones coinciden con las de la laptop en los 5,067 casos. Cuando pusimos a la placa a clasificar sin descanso, la fuente de alimentación y el calor limitaron su uso real tanto como el propio modelo. El sistema tiene fines educativos y no sustituye un diagnóstico médico.

**Palabras clave:** lesiones cutáneas, melanoma, dermatoscopia, aprendizaje por transferencia, MobileNetV2, cómputo en el borde, Raspberry Pi

# 1. Introducción

El melanoma ocupa un lugar discreto en las tablas de incidencia. En las de mortalidad pesa mucho más. GLOBOCAN estimó para 2024 que en el mundo hubo 337,969 casos nuevos de melanoma de piel y 58,398 muertes por esa causa [1], y para México la misma fuente calcula 2,461 casos nuevos y 809 defunciones en un año [2]. Importa reconocerlo a tiempo. Sin aparatos, la exactitud clínica para identificar un melanoma ronda el 60 % [3].

Un dermatoscopio deja ver la lesión con aumento y bajo luz polarizada. Kittler y sus coautores juntaron 27 estudios en un metaanálisis y encontraron que, en manos de examinadores con experiencia, la dermatoscopia eleva de forma significativa la capacidad diagnóstica respecto a la inspección a ojo desnudo (log odds ratio de 4.0 contra 2.7) [3]. Pero hay que tener el equipo y saber interpretarlo, y las dos cosas suelen quedarse en los consultorios especializados.

Esteva y colaboradores entrenaron una red neuronal convolucional con 129,450 imágenes clínicas y la compararon con 21 dermatólogos certificados en tareas de clasificación binaria; la red quedó a la par de los especialistas [4]. Poco después, Haenssle y su equipo enfrentaron una red entrenada con imágenes dermatoscópicas contra 58 dermatólogos. Con la misma sensibilidad, la red alcanzó una especificidad de 82.5 %, mayor que la de los médicos [5]. Sistemas así suelen correr en servidores con tarjetas gráficas, y fuera de los datos con que se entrenaron no siempre rinden igual. Con fotografías de pieles oscuras, por ejemplo, su desempeño cae de forma marcada [6].

Nosotros quisimos ver hasta dónde llega un sistema hecho con hardware barato y herramientas abiertas, sin servidor ni tarjeta gráfica. Elegimos MobileNetV2 [7], una arquitectura pensada para dispositivos móviles, la entrenamos con el conjunto ISIC 2019 [8] en una laptop para que distinguiera ocho tipos de lesión y la instalamos en una Raspberry Pi 5 con pantalla táctil. Aparte del modelo documentamos cuánto cambia el resultado al pasar de una computadora a otra y qué necesita la placa, en energía y en temperatura, para trabajar sin cortes. Pensamos el sistema como herramienta educativa. No está validado clínicamente ni pretende diagnosticar.

# 2. Resultados

El sistema completo aparece en la Fig. 1. En la laptop preparamos las imágenes y entrenamos la red. El archivo final viaja por el repositorio del proyecto hasta la Raspberry Pi, donde una aplicación lo usa para clasificar lo que capta la cámara. Las curvas y las métricas vienen de las bitácoras del entrenamiento y de la evaluación con el conjunto de prueba. Tiempos, voltajes y temperaturas los tomamos en la propia placa.

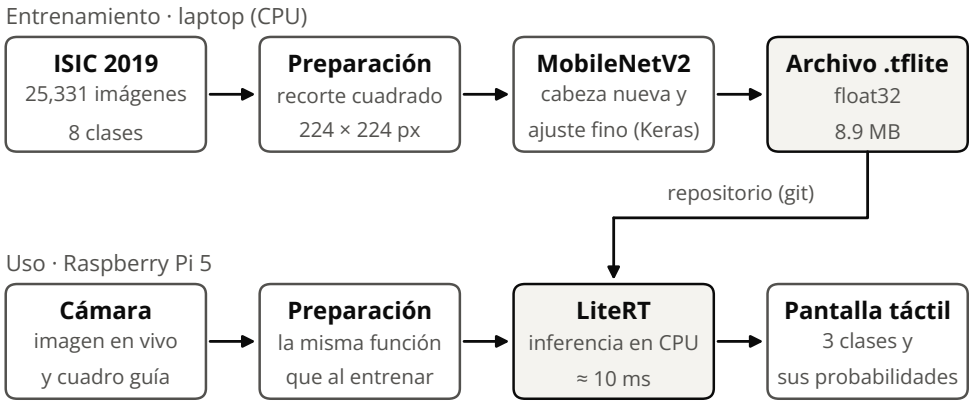



**Fig. 1** Arquitectura de Computer Vision DermaCheck. Las etapas de arriba corren una sola vez en la laptop, y las de abajo corresponden al uso diario en la Raspberry Pi 5. Ambos lados preparan la imagen con la misma función, así que en la placa la red recibe exactamente el tipo de entrada con el que aprendió.

## 2.1. Entrenamiento

Entrenar todo tomó 44.6 minutos en el CPU de una laptop con procesador Apple M4, sin aceleración gráfica. Lo corrimos de madrugada, con la computadora conectada al cargador. La primera fase fue lenta. En ella solo aprende la capa final, y tras seis épocas el recall promedio de validación llegaba a duras penas a 0.441 (Fig. 2). El cambio vino al descongelar las últimas 40 capas de MobileNetV2. En la primera época de esa fase la curva dio un traspié. Luego trepó rápido, y en la época 17, cuando el paso de aprendizaje se redujo a la mitad, el recall ganó más de nueve centésimas de un brinco. El mejor valor, 0.670, llegó en la época 19. Con esos pesos nos quedamos.

La pérdida de validación dejó de bajar cerca de 0.71, mientras que la de entrenamiento siguió hasta 0.53. Esa brecha, que se iba abriendo, es señal de que la red empezaba a memorizar detalles propios del conjunto de entrenamiento. Es un motivo más para no estirar la segunda fase mientras no tengamos más datos o más aumento de imágenes.

## 2.2. Desempeño en el conjunto de prueba

Las 5,067 imágenes de prueba quedaron apartadas desde el inicio y no intervinieron ni en el ajuste de los pesos ni en la elección de la mejor época. Las usamos una sola vez. El modelo acertó en 3,672 de ellas, una exactitud global de 0.725, y su recall promedio por clase fue de 0.632, un poco por debajo del 0.670 que había marcado en validación. Clase por clase, el resultado está en la Tabla 1.

El mejor recall fue para las lesiones vasculares (0.862) y los nevos (0.855). Las vasculares, rojizas y bien delimitadas, se dejan separar con facilidad. Al fondo, con los recalls más magros, quedaron el carcinoma escamoso (0.419), la queratosis actínica (0.451) y el melanoma (0.471), y la matriz de confusión de la Fig. 3 deja ver hacia

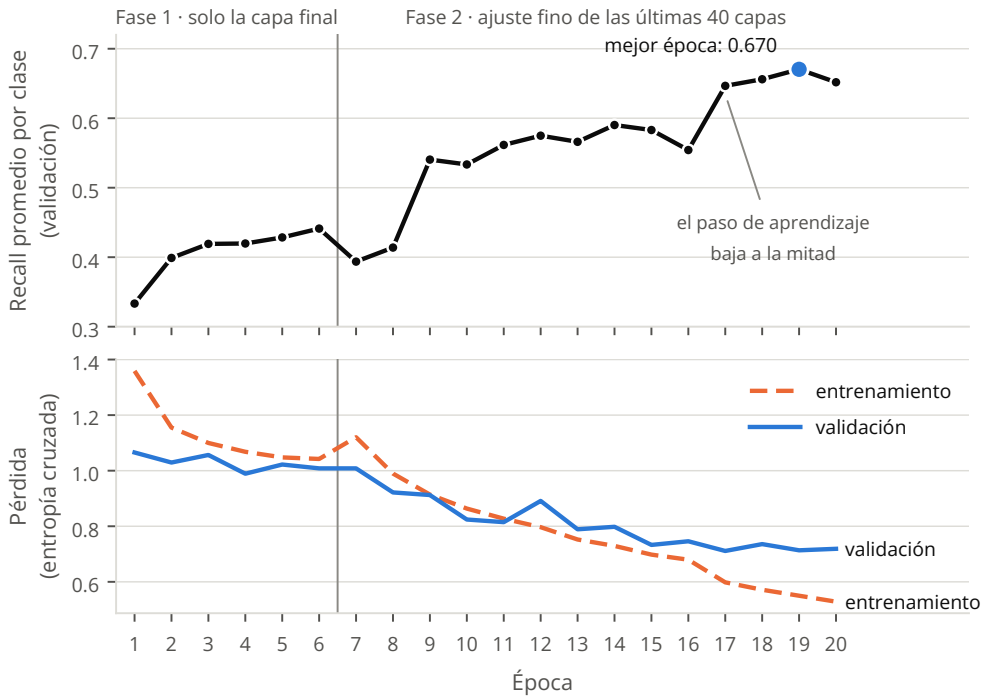



**Fig. 2** Evolución del entrenamiento. El panel superior muestra el recall promedio por clase en validación, la métrica con la que escogimos qué pesos conservar. En el inferior está la pérdida, tanto en entrenamiento como en validación. La línea vertical marca dónde termina la fase en que solo aprende la capa final y empieza el ajuste fino.

dónde se fueron esos errores. De los 949 melanomas, 298 (31.4 %) acabaron etiquetados como nevos y otros 111 (11.7 %) como queratosis benignas. Y un tercio de los carcinomas escamosos terminó confundido con carcinoma basocelular. Con el dermatofibroma pasa al revés. Su recall es bueno, pero la precisión se hunde a 0.234 porque el modelo lo sugiere a menudo para lesiones que no lo son, algo previsible si se piensa en el peso alto que le dimos a una clase tan escasa.

También juntamos las tres clases malignas (melanoma, carcinoma basocelular y carcinoma escamoso) frente a todo lo demás. Visto así, el modelo marcó con alguna de esas tres etiquetas 1,102 de las 1,672 lesiones malignas, una sensibilidad de 0.659. De las 3,395 no malignas descartó bien 3,072, lo que da una especificidad de 0.905.

## 2.3. Modelo completo y modelo ligero

Exportamos el modelo en dos versiones. La completa guarda los pesos en punto flotante de 32 bits (float32). La ligera los cuantiza a enteros de 8 bits, con lo que el archivo adelgaza de 8.9 a 2.5 MB. En la Tabla 2 están lado a lado. Con la float32, la diferencia máxima frente a las probabilidades del modelo original fue de $9.2 \times 10^{-6}$ a lo largo de las 5,067 imágenes. Para fines prácticos, es el mismo modelo. La ligera fue

**Tabla 1** Desempeño por clase en el conjunto de prueba, con el modelo float32. El recall es la parte de las lesiones de cada clase que el modelo reconoció. La precisión mide otra cosa, qué fracción de sus predicciones de esa clase fue correcta.

| Clase | Precisión | Recall | F1 | Imágenes |
|---|---|---|---|---|
| AK, queratosis actínica | 0.425 | 0.451 | 0.437 | 182 |
| BCC, carcinoma basocelular | 0.665 | 0.792 | 0.723 | 606 |
| BKL, queratosis benigna | 0.483 | 0.615 | 0.541 | 507 |
| DF, dermatofibroma | 0.234 | 0.595 | 0.336 | 42 |
| MEL, melanoma | 0.761 | 0.471 | 0.582 | 949 |
| NV, nevo melanocítico | 0.850 | 0.855 | 0.852 | 2,606 |
| SCC, carcinoma escamoso | 0.422 | 0.419 | 0.421 | 117 |
| VASC, lesión vascular | 0.667 | 0.862 | 0.752 | 58 |
| Promedio por clase | 0.563 | 0.632 | 0.580 | 5,067 |
| Promedio ponderado | 0.742 | 0.725 | 0.725 | 5,067 |

Exactitud global: 0.725. En los promedios ponderados cada clase cuenta según su número de imágenes, y por eso se inclinan hacia lo que pasa con los nevos, que son la mitad del conjunto.

menos fiel. Eligió la misma clase que el original en el 94.5 % de las imágenes y cedió entre uno y 1.4 puntos de recall promedio.

**Tabla 2** Las dos versiones exportadas, frente a frente. Las métricas se calcularon con las 5,067 imágenes de prueba, y los tiempos en la Raspberry Pi 5 se tomaron clasificando las imágenes de una en una.

| | float32 | Ligero (int8) |
|---|---|---|
| Tamaño del archivo | 8.9 MB | 2.5 MB |
| Exactitud | 0.725 | 0.721–0.722[1] |
| Recall promedio por clase | 0.632 | 0.619–0.623[1] |
| Misma clase que el modelo de Keras | 100 % | 94.5 % |
| Tiempo por imagen en la Pi, 1 hilo | 21.7 ms | 19.0 ms |
| Tiempo por imagen en la Pi, 4 hilos | 9.7 ms | 11.5 ms |

En la Pi, con 2 hilos, la versión float32 tardó 12.7 ms por imagen.

[1] El primer valor es de la laptop que exportó el modelo y el segundo, de la Raspberry Pi. La diferencia se comenta en el texto.

Aquí nos llevamos una sorpresa. Entre la Pi y la laptop, las probabilidades de la float32 no se separaron ni tres millonésimas. En la ligera sí hubo distancia. La laptop que hizo la exportación dio un recall promedio de 0.619 y la Pi, uno de 0.623. En las imágenes de muestra, las probabilidades llegaron a diferir en 0.015. Sospechamos de las rutinas de cálculo entero que el intérprete elige según el procesador, porque esa misma laptop, con una versión anterior del intérprete, calca los resultados de la Pi. Como encima la ligera fue más lenta con cuatro hilos, la aplicación se quedó con la float32.

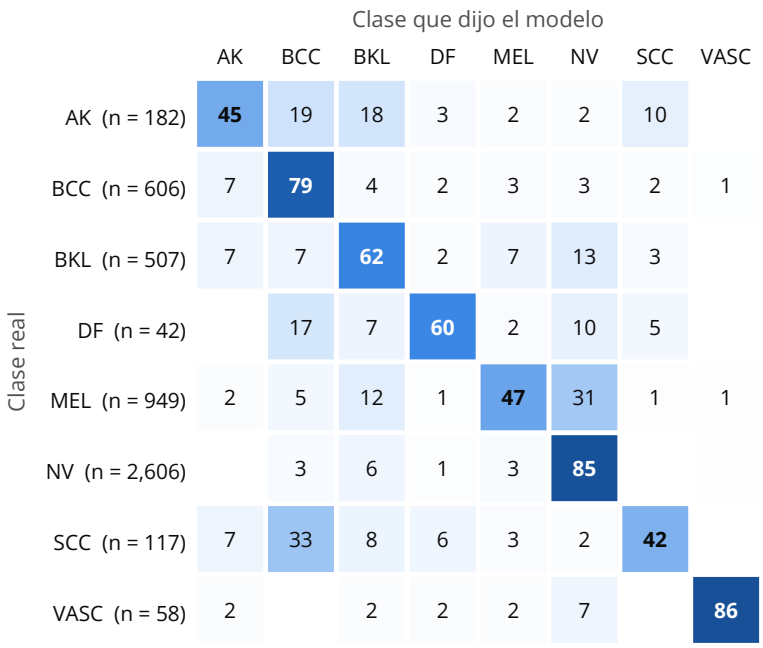



**Fig. 3** Matriz de confusión del conjunto de prueba, normalizada por filas. Cada celda dice qué porcentaje de las lesiones de una clase real recibió cada etiqueta, y los aciertos caen en la diagonal. Con $n$ se indica cuántas imágenes tiene cada clase. No se muestran los valores menores a 0.5 %.

## 2.4. Funcionamiento en la Raspberry Pi 5

En la propia Pi clasificamos las 5,067 imágenes de prueba, una por una, igual que lo hace la aplicación. La versión float32 no tuvo una sola discrepancia con la laptop en la clase elegida. Con los cuatro núcleos trabajando, cada imagen tomó 9.7 ms, así que en teoría cabrían unas cien por segundo. Ese ritmo no hace falta. La aplicación clasifica nada más cuando alguien toca el botón, y lo que tarda en responder depende más de la interfaz que de la red.

El verdadero estorbo fue la alimentación. En los primeros días la placa se nos apagó dos veces sin dar aviso, una de ellas en reposo y por una causa que no llegamos a aclarar. Volvimos a correr la clasificación completa, esta vez con un registro cada dos segundos de la temperatura, del voltaje en la línea de 5 V y de las alertas del firmware, y lo que encontramos está en la Tabla 3.

Alimentada desde el puerto USB-C de una laptop, la Pi aguantó con uno y con dos núcleos. Al arrancar los cuatro, el voltaje cayó a 4.70 V, el sistema anotó una alerta de bajo voltaje y dos segundos después la placa estaba apagada. Probamos entonces con un cargador de pared de 65 W. También negocia solo 3 A a 5 V, y aun así sostuvo unos cuatro minutos de carga casi continua, con muy poco margen, pues hubo siete alertas breves de bajo voltaje. El calor llegó después. Sin disipador, la placa alcanzó los 87.3 °C, bajó su frecuencia hasta 1.5 GHz durante cerca de 48 segundos y el tiempo por imagen subió de 9.7 a 11.9 ms. Si se suman los rieles internos que reporta el controlador de energía, el consumo rondó los 2 W en reposo y los 7 W con

**Tabla 3** Pruebas de carga en la Raspberry Pi 5, sin ventilador y clasificando sin pausa el conjunto de prueba. Las dos fuentes negociaron 3 A a 5 V, cuando la fuente oficial de la placa da 5 A.

| Alimentación | Hilos | Temp. máx. | 5 V mín. | Resultado |
|---|---|---|---|---|
| USB-C de laptop | 1 | 67.5 °C | 4.81 V | Estable |
| USB-C de laptop | 2 | 84.5 °C | 4.78 V | Estable, con freno térmico |
| USB-C de laptop | 4 | –[1] | 4.70 V | Se apagó a los 2 s |
| Cargador de 65 W | 4 | 87.3 °C | 4.76 V | Estable, 7 avisos de voltaje |

[1] La placa se apagó antes de que la temperatura alcanzara a subir. La última lectura marcó 75.2 °C.

los cuatro núcleos ocupados. Para este modelo, el fabricante recomienda una fuente de 5 A [9, 10].

### 2.5. La aplicación

La aplicación tiene dos pantallas, y la Fig. 4 las muestra al tamaño de la pantalla táctil de siete pulgadas que vamos a usar ($1024 \times 600$ píxeles). Las capturas las tomamos en un navegador ajustado a esa resolución. Todavía no hemos probado la aplicación en la pantalla física ni con la cámara. En la primera, la imagen en vivo se ve oscurecida, salvo un cuadro central con retícula, y lo que cae dentro de ese cuadro es justo lo que se clasifica. Al tocar «Tomar» aparece la segunda, con la clase más probable y su porcentaje, una escala de 0 a 100 donde se ubican las tres primeras clases y un aviso permanente de que el resultado no es un diagnóstico. Cuando alguna clase de riesgo (melanoma, carcinomas o queratosis actínica) llega a 25 % dentro de las tres primeras, salta en ámbar la sugerencia de ver a un dermatólogo. Si la clase principal se queda por debajo de 50 %, la aplicación advierte que el resultado es poco seguro.

La demostración puede hacerse incluso sin cámara. Para eso la aplicación trae un modo de prueba con 62 fotografías de licencia abierta. De ellas, 40 son dermatoscópicas del propio ISIC, cinco por clase escogidas al azar, y las otras 22 son fotos clínicas de Wikimedia Commons, tomadas con cámaras comunes. Con las dermatoscópicas, el modelo dio con la clase correcta en 25 de 40. Con las clínicas, solo en 6 de 22. Son muestras pequeñas, pero la caída es la que cabía esperar de un modelo que aprendió únicamente con imágenes de dermatoscopio.

## 3. Métodos

### 3.1. Conjunto de datos

La base fue el conjunto de entrenamiento del reto ISIC 2019, con 25,331 imágenes dermatoscópicas de ocho categorías diagnósticas [8, 11]. En él se juntan tres colecciones. De HAM10000 vienen 10,015 imágenes [12], y el resto se reparte entre BCN20000, del Hospital Clínic de Barcelona [13], y el reto ISIC 2017 [14]. La copia con la que trabajamos está publicada en Hugging Face y ya viene partida en 20,264 imágenes para entrenar y 5,067 para probar [15]. Esas dos partes juntas dan las 25,331 del conjunto de entrenamiento oficial. Lo que aquí llamamos prueba, entonces, es una porción

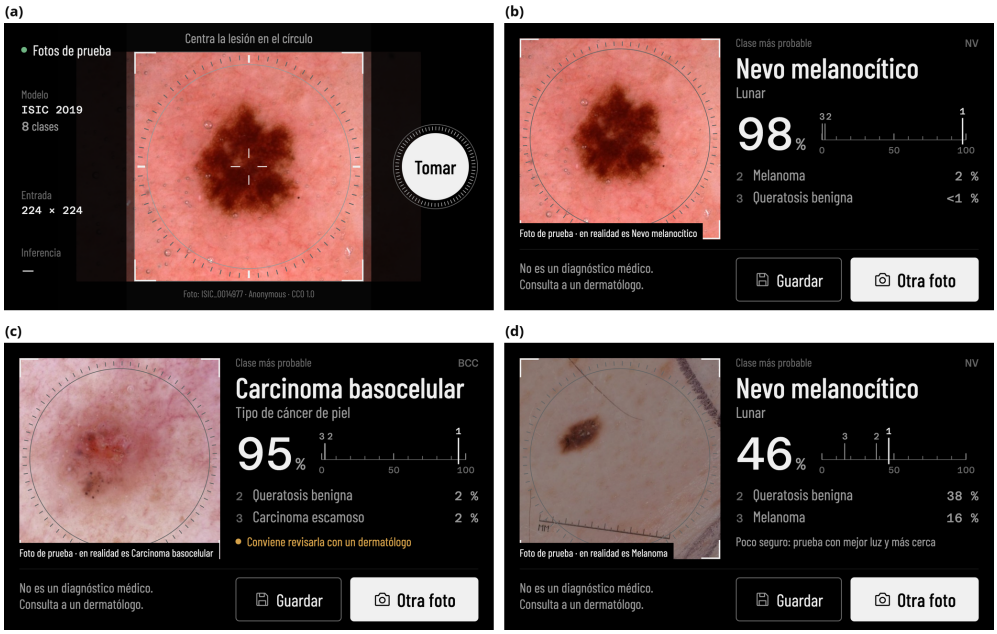



**Fig. 4** Capturas de la aplicación con imágenes de prueba de ISIC. (a) Pantalla de cámara, donde el cuadro guía marca la región que se clasifica. (b) Un nevo, reconocido con 98 %. (c) Carcinoma basocelular reconocido con 95 %, junto al aviso ámbar que sugiere ir al dermatólogo. (d) Melanoma que el modelo toma por nevo, con apenas 46 % y la advertencia de resultado poco seguro.

apartada de ese mismo conjunto y no la prueba oficial del reto. De las 20,264 reservamos un 10 % estratificado para validación (2,027), y los pesos se ajustaron con las 18,237 que quedaron. La Tabla 4 desglosa las particiones por clase y la Fig. 5 enseña una imagen de cada una.

## 3.2. Preparación de las imágenes

Antes de llegar a la red, toda imagen pasa por una misma función. Esta la endereza según la orientación anotada en sus metadatos EXIF, la convierte a RGB, le corta el cuadrado central más grande que quepa y lo reduce a $224 \times 224$ píxeles con interpolación bilineal. En la Raspberry Pi vive una copia de esa función, y una prueba automática revisa que las dos entreguen exactamente los mismos píxeles. A la red le entran valores de 0 a 255, sin normalizar. El cambio al intervalo que espera MobileNetV2 lo hace el propio modelo,

$$\tilde{x} = \frac{x}{127.5} - 1, \tag{1}$$

y así el programa de la placa se ahorra ese paso. Durante el entrenamiento cada imagen recibía alteraciones al azar: volteos horizontales y verticales, rotaciones de hasta media vuelta hacia cualquier lado, cambios de escala de hasta 15 % y variaciones de brillo y

**Tabla 4** Imágenes de cada clase en las tres particiones.

| Clase | Entrenamiento | Validación | Prueba | Total |
|---|---|---|---|---|
| AK, queratosis actínica | 616 | 69 | 182 | 867 |
| BCC, carcinoma basocelular | 2,445 | 272 | 606 | 3,323 |
| BKL, queratosis benigna | 1,905 | 212 | 507 | 2,624 |
| DF, dermatofibroma | 177 | 20 | 42 | 239 |
| MEL, melanoma | 3,216 | 357 | 949 | 4,522 |
| NV, nevo melanocítico | 9,242 | 1,027 | 2,606 | 12,875 |
| SCC, carcinoma escamoso | 460 | 51 | 117 | 628 |
| VASC, lesión vascular | 176 | 19 | 58 | 253 |
| Total | 18,237 | 2,027 | 5,067 | 25,331 |

Bajo BKL caen el lentigo solar, la queratosis seborreica y la queratosis liquenoide. El reparto es disparejo en extremo, con 52 nevos por cada dermatofibroma en la partición de entrenamiento.

contraste de esa misma magnitud. Esas alteraciones solo existen mientras se entrena. El modelo exportado no carga con ellas.

## 3.3. Arquitectura y entrenamiento

La pieza con que se arma MobileNetV2 es el bloque residual invertido. Cada uno toma una representación compacta, la ensancha, la filtra con convoluciones separables en profundidad y la vuelve a estrechar con una proyección lineal [7, 16]. Cargamos la red con los pesos que aprendió en ImageNet [17], le quitamos su clasificador original y en su lugar montamos un promedio global, una capa de *dropout* con tasa 0.3 y una capa densa de ocho salidas con activación softmax. En total, 2,268,232 parámetros. ¿De qué sirven unos pesos aprendidos con fotos de perros, coches y otros objetos ajenos a la piel? Las primeras capas de una red entrenada con imágenes naturales captan rasgos generales, como bordes y manchas de color, que valen también para otras tareas [18].

Entrenamos en dos fases, las dos con el optimizador Adam [19] y lotes de 64 imágenes. Durante las primeras seis épocas la base estuvo congelada y solo se actualizaban los 10,248 parámetros de la capa final, con un paso de aprendizaje de $10^{-3}$. Después descongelamos las últimas 40 capas de MobileNetV2, aunque dejamos fijas las de normalización por lotes [20] para no perder las estadísticas que traían de ImageNet. Así subieron a 1,673,608 los parámetros entrenables, ajustados con un paso de $10^{-4}$ durante 14 épocas como máximo. Trabajamos con TensorFlow 2.21 y Keras 3.15 [21, 22].

Contra el desbalance [23] ponderamos la pérdida por clase. Cada imagen de la clase $c$ entra con un peso

$$w_c = \left( \frac{N}{K \, n_c} \right)^{1/2}, \tag{2}$$

donde $N$ es el número de imágenes de entrenamiento, $K = 8$ el número de clases y $n_c$ las imágenes de la clase $c$. La raíz cuadrada amortigua el efecto, y con ella los pesos van de 0.50 en los nevos a 3.60 en las lesiones vasculares. La proporción inversa a secas los habría estirado de 0.25 a 12.95. Lo que se minimiza es la entropía cruzada

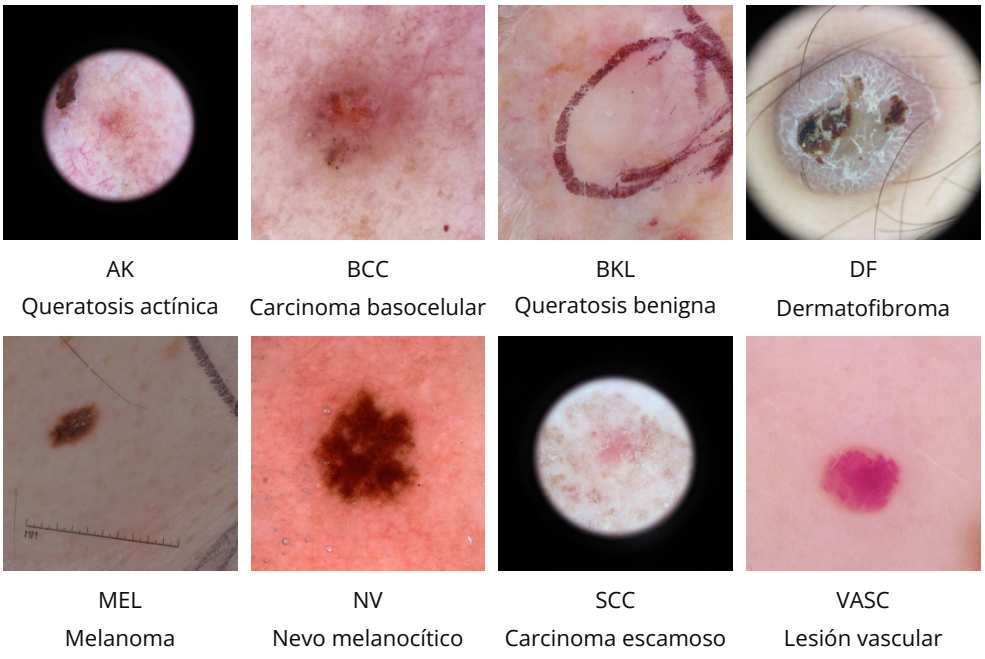



**Fig. 5** Una imagen de prueba por clase, con el mismo recorte cuadrado que le llega a la red. El crédito y la licencia de cada una están en el Apéndice A.

ponderada, promediada sobre cada lote de $B$ imágenes,

$$\mathcal{L} = -\frac{1}{B}\sum_{i=1}^{B} w_{y_i} \log p_{i,y_i}, \tag{3}$$

con $y_i$ la clase verdadera de la imagen $i$ y $p_{i,y_i}$ la probabilidad que el modelo le asigna a esa clase. Los hiperparámetros están reunidos en la Tabla 5.

**Tabla 5** Hiperparámetros de las dos fases del entrenamiento.

| Parámetro | Fase 1 | Fase 2 |
|---|---|---|
| Capas entrenables | solo la capa final | últimas 40 de la base y la capa final |
| Paso de aprendizaje inicial | $10^{-3}$ | $10^{-4}$ |
| Épocas máximas | 6 | 14 |
| Parámetros entrenables | 10,248 | 1,673,608 |
| Tamaño de lote | 64 | |
| Optimizador | Adam | |
| *Dropout* | 0.3 | |
| Parada temprana | 4 épocas sin mejora del recall promedio | |
| Reducción del paso | a la mitad tras 2 épocas sin mejora | |

## 3.4. Selección del modelo y métricas

Con la exactitud global es fácil engañarse. La mitad de las imágenes son nevos, y un modelo que respondiera «nevo» a todo ya acertaría la mitad de las veces sin distinguir nada. Por eso escogimos los pesos con el recall promedio por clase en validación,

$$\bar{R} = \frac{1}{K} \sum_{c=1}^{K} \frac{\mathrm{VP}_c}{\mathrm{VP}_c + \mathrm{FN}_c}, \tag{4}$$

donde $\mathrm{VP}_c$ son las imágenes de la clase $c$ bien clasificadas y $\mathrm{FN}_c$ las que acabaron con otra etiqueta. Calculábamos $\bar{R}$ al cerrar cada época y solo guardábamos los pesos cuando subía. Y esa misma cifra gobernaba la parada temprana y la reducción del paso de aprendizaje. En la prueba reportamos además la precisión, $\mathrm{VP}_c/(\mathrm{VP}_c + \mathrm{FP}_c)$, y la puntuación F1, media armónica entre precisión y recall.

## 3.5. Exportación a TensorFlow Lite

Convertimos el modelo a TensorFlow Lite, el formato que Google rebautizó como LiteRT en 2024 [24, 25], y sacamos dos versiones, una en float32 y otra ligera. La ligera usa cuantización de rango dinámico. Guarda los pesos como enteros de 8 bits, que se vuelven reales con la relación afín

$$r = S\,(q - Z), \tag{5}$$

en la que $q$ es el entero almacenado, $S$ una escala y $Z$ el entero que corresponde al cero [26]. Las activaciones, en cambio, las cuantiza el intérprete al vuelo durante la inferencia, y de ahí viene que el resultado cambie un poco de un procesador a otro. Con cada modelo van 16 imágenes de prueba y las probabilidades que la laptop les asignó. Al instalarse el modelo, la Pi clasifica esas imágenes y coteja sus números con los de la laptop. Por aquella variación, a la versión ligera le damos más holgura, con una tolerancia de 0.05 contra $10^{-3}$ para la float32.

## 3.6. Hardware y aplicación

La placa es una Raspberry Pi 5 de 8 GB de memoria, con un procesador Broadcom BCM2712 de cuatro núcleos Arm Cortex-A76 a 2.4 GHz [27]. Corre Raspberry Pi OS de 64 bits, Python 3.13 y el intérprete `ai-edge-litert` [28]. La pantalla táctil es un modelo genérico de siete pulgadas y $1024 \times 600$ píxeles, conectado por HDMI. Por dentro, la aplicación es un servidor pequeño en Python, que lee la cámara y ejecuta el modelo, más una página que Chromium muestra a pantalla completa. La imagen puede venir del módulo de cámara de la Raspberry Pi o de una cámara web USB, y si no hay ninguna conectada, la aplicación usa las fotografías del modo de prueba. Una foto solo se guarda si el usuario lo pide. El Algoritmo 1 detalla lo que pasa al tocar «Tomar».

Para medir el rendimiento pasamos por la placa las 5,067 imágenes de prueba, cada una por separado como en la aplicación, con uno, dos y cuatro hilos. Mientras

**Algoritmo 1** Qué hace la aplicación al tocar «Tomar»

**Entrada:** cuadro más reciente de la cámara $I$ de $w \times h$ píxeles; modelo $M$; fracción del cuadro guía $g = 0.8$
**Salida:** las tres clases más probables y los avisos que se muestran

$s \leftarrow \mathrm{round}(g \cdot \mathrm{mín}(w, h))$
$C \leftarrow$ cuadrado de lado $s$ centrado en $I$ ▷ la región del cuadro guía
$x \leftarrow C$ reducido a $224 \times 224$ píxeles con interpolación bilineal
$p \leftarrow M(x)$ ▷ ocho probabilidades que suman 1
$T \leftarrow$ las tres clases con mayor $p$
**si** alguna clase de $T$ es MEL, BCC, SCC o AK y su probabilidad es al menos 0.25 **entonces**
    mostrar el aviso ámbar «Conviene revisarla con un dermatólogo»
**fin si**
**si** $\max p < 0.5$ **entonces**
    mostrar «Poco seguro: prueba con mejor luz y más cerca»
**fin si**
**devolver** $T$ con sus probabilidades

tanto, un proceso aparte anotaba cada dos segundos la temperatura del procesador, el voltaje de la línea de 5 V que reporta el controlador de energía y las alertas del firmware por bajo voltaje o por frecuencia reducida. Ese mismo proceso copiaba los mensajes del sistema a la tarjeta de memoria, para no perderlos si la placa volvía a apagarse.

**Declaración ética.** En el estudio no participaron personas ni se usaron muestras humanas. Todas las imágenes vienen de colecciones públicas y anonimizadas, cuyas licencias autorizan el uso académico, y se citan con su crédito.

## 4. Discusión

El modelo reconoce menos de la mitad de los melanomas. Ningún otro resultado nos incomoda tanto, ni importa tanto. Cuando falla con esa clase, la mayoría de las veces toma la lesión por un nevo, el desliz más dañino de todos, porque deja tranquila a una persona que tendría que ir a consulta. La precisión para el melanoma no está mal (0.761). Si la red dice «melanoma», suele tener razón, pero lo dice demasiado poco. Una parte de la culpa la tienen los pesos de la Ecuación (2). Con la raíz cuadrada, al melanoma le tocó un peso de 0.84, menor que el de clases mucho más raras, y la red acabó inclinada a no proponerlo. Mover el umbral de decisión, o darle al melanoma un peso propio, podría cambiar algo de precisión por sensibilidad. En una herramienta como esta, ese trueque conviene.

Poner nuestras cifras al lado de las de la literatura pide varias salvedades. Los grupos de Esteva y de Haenssle evaluaron tareas binarias, con otros conjuntos de imágenes y con redes más grandes [4, 5]. La nuestra decide entre ocho clases muy desbalanceadas y cabe en un archivo de 9 MB. Otra reserva pesa en contra nuestra. La partición entre entrenamiento y prueba no separa por lesión, y HAM10000 trae varias fotos de una misma lesión. Si algunas cayeron a los dos lados, nuestras cifras

pecan un poco de optimistas. Aun así, que una red de 2.3 millones de parámetros, entrenada en 45 minutos y sin tarjeta gráfica, separe nevos, carcinomas basocelulares y lesiones vasculares con un recall superior a 0.79 dice mucho de lo accesible que se volvió esta tecnología.

En la Raspberry Pi, la red fue lo más llevadero, con menos de 10 ms por imagen. Los problemas vinieron de la alimentación y del calor. Conectada al USB-C de una laptop, la placa se apagó en cuanto le pedimos los cuatro núcleos, y sin disipador la temperatura la frena en uno o dos minutos. Para una demostración sin sobresaltos alcanza con la fuente oficial de 27 W y un disipador activo. Si no están a mano, queda limitar el modelo a dos hilos, que a 12.7 ms por imagen sigue sobrado de velocidad.

La limitación más seria está en las imágenes con las que aprendió. Todas venían de dermatoscopio, tomadas pegadas a la piel y con luz controlada, y en el modo de prueba el modelo acertó 25 de 40 imágenes dermatoscópicas pero no pasó de 6 aciertos en 22 fotos clínicas. Con una cámara conectada a la Pi todavía no lo probamos, aunque lo más probable es que ocurra algo parecido. Y lo mismo debería pasar con tonos de piel poco representados en ISIC, una carencia que ya se documentó en otros sistemas [6]. Acoplar a la cámara un lente dermatoscópico económico y reentrenar con imágenes más diversas serían los siguientes pasos. Hay un pendiente aún más elemental. Hoy el modelo siempre escoge una de las ocho clases, aunque le pongan enfrente la foto de una mano o de una taza, así que necesita alguna forma de reconocer cuando la imagen no muestra ninguna lesión.

## 5. Conclusión

Nuestro clasificador de lesiones cutáneas pasó de la laptop donde lo entrenamos a una placa de bajo costo sin perder exactitud en el traslado. La Raspberry Pi repitió, imagen por imagen, las predicciones originales. Frente a un melanoma, sin embargo, Computer Vision DermaCheck todavía no es de fiar, y por eso la aplicación insiste en que no diagnostica. Lo que vale del prototipo es otra cosa. En un salón de clases vuelve tangible cómo aprende una red neuronal y por qué se equivoca donde se equivoca, y deja medido qué hace falta para sacarla de una computadora y meterla en un aparato que cabe en la mano.

**Información suplementaria.** En el repositorio del proyecto, disponible a solicitud de los autores, guardamos el código, los cuadernos de entrenamiento y de evaluación, el modelo exportado, los registros de las pruebas en la Raspberry Pi y los créditos de las 62 imágenes del modo de prueba.

**Agradecimientos.** Agradecemos a la International Skin Imaging Collaboration y a las instituciones que donaron las imágenes de ISIC 2019. La copia del conjunto con la que trabajamos se la debemos a quien la publicó en Hugging Face, y las fotos clínicas del modo de prueba, a los fotógrafos de Wikimedia Commons que las compartieron con licencia abierta.

## Declaraciones

- **Financiamiento:** el proyecto no recibió financiamiento. El hardware corrió por cuenta del equipo.
- **Conflicto de intereses:** los autores no tienen conflictos de intereses que declarar.
- **Aprobación ética y consentimiento para participar:** no aplica, porque solo se usaron imágenes públicas y anonimizadas.
- **Consentimiento para publicación:** no aplica.
- **Disponibilidad de datos:** el conjunto ISIC 2019 se obtiene en el sitio del reto [11], y la copia que usamos está en Hugging Face [15]. Las métricas y los registros de este trabajo se guardan en el repositorio del proyecto.
- **Disponibilidad de materiales:** no aplica.
- **Disponibilidad del código:** disponible a solicitud de los autores.
- **Contribución de los autores:** los cuatro autores participaron en todas las etapas, desde la concepción del proyecto y la preparación de los datos hasta el entrenamiento y la evaluación del modelo, el armado del prototipo y la redacción del manuscrito. Todos aprobaron la versión final.

## Apéndice A Créditos de las imágenes

Las imágenes de la Fig. 5 y de la Fig. 4 provienen del archivo de ISIC. En la Tabla A1 va el identificador de cada una, con el autor y la licencia tal como los registra el propio archivo.

**Tabla A1** Autoría y licencia de las imágenes de ISIC que aparecen en las figuras.

| Identificador | Clase | Autor | Licencia |
|---|---|---|---|
| ISIC_0062738 | AK | Hospital Clínic de Barcelona | CC BY-NC 4.0 |
| ISIC_0029546 | BCC | MILK study team | CC BY-NC 4.0 |
| ISIC_0012323 | BKL | Anónimo | CC0 1.0 |
| ISIC_0071844 | DF | Hospital Clínic de Barcelona | CC BY-NC 4.0 |
| ISIC_0012425 | MEL | Anónimo | CC0 1.0 |
| ISIC_0014977 | NV | Anónimo | CC0 1.0 |
| ISIC_0071727 | SCC | Hospital Clínic de Barcelona | CC BY-NC 4.0 |
| ISIC_0029439 | VASC | MILK study team | CC BY-NC 4.0 |

En la Fig. 4 aparecen ISIC_0014977, ISIC_0029546 e ISIC_0012425.

## Referencias

[1] Sung, H., Filho, A.M., Laversanne, M., Ferlay, J., Siegel, R.L., Soerjomataram, I., Jemal, A., Bray, F.: Global cancer statistics 2024: GLOBOCAN estimates of incidence and mortality worldwide for 34 cancers in 186 countries. CA: A Cancer Journal for Clinicians **76**(4), 70090 (2026) https://doi.org/10.3322/caac.70090

[2] Ervik, M., Laversanne, M., Lam, F., Colombet, M., Ferlay, J., Filho, A.M., Bray, F.: Global Cancer Observatory: Cancer Today (2024 estimates, version 1.0). Mexico: fact sheet. International Agency for Research on Cancer, Lyon, Francia. Consultado el 10 de noviembre de 2026 (2026). https://gco.iarc.who.int/media/globocan/factsheets/populations/484-mexico-fact-sheet.pdf

[3] Kittler, H., Pehamberger, H., Wolff, K., Binder, M.: Diagnostic accuracy of dermoscopy. The Lancet Oncology **3**(3), 159–165 (2002) https://doi.org/10.1016/S1470-2045(02)00679-4

[4] Esteva, A., Kuprel, B., Novoa, R.A., Ko, J., Swetter, S.M., Blau, H.M., Thrun, S.: Dermatologist-level classification of skin cancer with deep neural networks. Nature **542**(7639), 115–118 (2017) https://doi.org/10.1038/nature21056

[5] Haenssle, H.A., Fink, C., Schneiderbauer, R., Toberer, F., Buhl, T., Blum, A., Kalloo, A., Hassen, A.B.H., Thomas, L., Enk, A., Uhlmann, L., Reader study level-I and level-II Groups: Man against machine: diagnostic performance of a deep learning convolutional neural network for dermoscopic melanoma recognition in comparison to 58 dermatologists. Annals of Oncology **29**(8), 1836–1842 (2018) https://doi.org/10.1093/annonc/mdy166

[6] Daneshjou, R., Vodrahalli, K., Novoa, R.A., Jenkins, M., Liang, W., Rotemberg, V., Ko, J., Swetter, S.M., Bailey, E.E., Gevaert, O., Mukherjee, P., Phung, M., Yekrang, K., Fong, B., Sahasrabudhe, R., Allerup, J.A.C., Okata-Karigane, U., Zou, J., Chiou, A.S.: Disparities in dermatology AI performance on a diverse, curated clinical image set. Science Advances **8**(32), 6147 (2022) https://doi.org/10.1126/sciadv.abq6147

[7] Sandler, M., Howard, A., Zhu, M., Zhmoginov, A., Chen, L.-C.: MobileNetV2: Inverted residuals and linear bottlenecks. In: 2018 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), pp. 4510–4520. IEEE, Salt Lake City, UT, USA (2018). https://doi.org/10.1109/CVPR.2018.00474

[8] International Skin Imaging Collaboration: ISIC 2019 Challenge. Consultado el 10 de noviembre de 2026 (2019). https://challenge.isic-archive.com/landing/2019/

[9] Raspberry Pi Ltd: Raspberry Pi hardware: Power supply. Raspberry Pi Documentation. Consultado el 10 de noviembre de 2026. https://www.raspberrypi.com/documentation/computers/raspberry-pi.html#power-supply

[10] Raspberry Pi Ltd: Raspberry Pi 27W USB-C Power Supply. Consultado el 10 de noviembre de 2026. https://www.raspberrypi.com/products/27w-power-supply/

[11] International Skin Imaging Collaboration: ISIC Challenge Datasets: ISIC 2019. Consultado el 10 de noviembre de 2026. https://challenge.isic-archive.com/data/

[12] Tschandl, P., Rosendahl, C., Kittler, H.: The HAM10000 dataset, a large collection of multi-source dermatoscopic images of common pigmented skin lesions. Scientific Data **5**, 180161 (2018) https://doi.org/10.1038/sdata.2018.161

[13] Hernández-Pérez, C., Combalia, M., Podlipnik, S., Codella, N.C.F., Rotemberg, V., Halpern, A.C., Reiter, O., Carrera, C., Barreiro, A., Helba, B., Puig, S., Vilaplana, V., Malvehy, J.: BCN20000: Dermoscopic lesions in the wild. Scientific Data **11**(1), 641 (2024) https://doi.org/10.1038/s41597-024-03387-w

[14] Codella, N.C.F., Gutman, D., Celebi, M.E., Helba, B., Marchetti, M.A., Dusza, S.W., Kalloo, A., Liopyris, K., Mishra, N., Kittler, H., Halpern, A.: Skin lesion analysis toward melanoma detection: A challenge at the 2017 International Symposium on Biomedical Imaging (ISBI), hosted by the International Skin Imaging Collaboration (ISIC). In: 2018 IEEE 15th International Symposium on Biomedical Imaging (ISBI 2018), pp. 168–172. IEEE, Washington, DC, USA (2018). https://doi.org/10.1109/ISBI.2018.8363547

[15] Atoany: ISIC2019. Conjunto de datos en Hugging Face. Consultado el 10 de noviembre de 2026 (2025). https://huggingface.co/datasets/Atoany/ISIC2019

[16] Howard, A.G., Zhu, M., Chen, B., Kalenichenko, D., Wang, W., Weyand, T., Andreetto, M., Adam, H.: MobileNets: Efficient Convolutional Neural Networks for Mobile Vision Applications. arXiv preprint arXiv:1704.04861 (2017). https://doi.org/10.48550/arXiv.1704.04861 . https://arxiv.org/abs/1704.04861

[17] Deng, J., Dong, W., Socher, R., Li, L.-J., Li, K., Fei-Fei, L.: ImageNet: A large-scale hierarchical image database. In: 2009 IEEE Conference on Computer Vision and Pattern Recognition (CVPR), pp. 248–255. IEEE, Miami, FL, USA (2009). https://doi.org/10.1109/CVPR.2009.5206848

[18] Yosinski, J., Clune, J., Bengio, Y., Lipson, H.: How transferable are features in deep neural networks? In: Ghahramani, Z., Welling, M., Cortes, C., Lawrence, N., Weinberger, K. (eds.) Advances in Neural Information Processing Systems, vol. 27, pp. 3320–3328. Curran Associates, Inc., Red Hook, NY, USA (2014). arXiv:1411.1792. https://arxiv.org/abs/1411.1792

[19] Kingma, D.P., Ba, J.: Adam: A method for stochastic optimization. In: 3rd International Conference on Learning Representations (ICLR 2015), San Diego, CA, USA (2015). arXiv:1412.6980. https://arxiv.org/abs/1412.6980

[20] Ioffe, S., Szegedy, C.: Batch normalization: Accelerating deep network training by reducing internal covariate shift. In: Bach, F., Blei, D. (eds.) Proceedings of the 32nd International Conference on Machine Learning. Proceedings of Machine Learning Research, vol. 37, pp. 448–456. PMLR, Lille, France (2015). https://proceedings.mlr.press/v37/ioffe15.html

[21] Abadi, M., Barham, P., Chen, J., Chen, Z., Davis, A., Dean, J., Devin, M., Ghemawat, S., Irving, G., Isard, M., Kudlur, M., Levenberg, J., Monga, R., Moore, S., Murray, D.G., Steiner, B., Tucker, P., Vasudevan, V., Warden, P., Wicke, M., Yu, Y., Zheng, X.: TensorFlow: A system for large-scale machine learning. In: 12th USENIX Symposium on Operating Systems Design and Implementation (OSDI 16), pp. 265–283. USENIX Association, Savannah, GA, USA (2016). https://www.usenix.org/conference/osdi16/technical-sessions/presentation/abadi

[22] Chollet, F., et al.: Keras. Consultado el 10 de noviembre de 2026 (2015). https://keras.io

[23] He, H., Garcia, E.A.: Learning from imbalanced data. IEEE Transactions on Knowledge and Data Engineering **21**(9), 1263–1284 (2009) https://doi.org/10.1109/TKDE.2008.239

[24] Google AI Edge: LiteRT: High-Performance On-Device Machine Learning Framework. Consultado el 10 de noviembre de 2026 (2026). https://developers.google.com/edge/litert

[25] Google AI Edge Team: TensorFlow Lite is now LiteRT. Google for Developers Blog. Consultado el 10 de noviembre de 2026 (2024). https://developers.googleblog.com/en/tensorflow-lite-is-now-litert/

[26] Jacob, B., Kligys, S., Chen, B., Zhu, M., Tang, M., Howard, A., Adam, H., Kalenichenko, D.: Quantization and training of neural networks for efficient integer-arithmetic-only inference. In: 2018 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), pp. 2704–2713. IEEE, Salt Lake City, UT, USA (2018). https://doi.org/10.1109/CVPR.2018.00286

[27] Raspberry Pi Ltd: Raspberry Pi 5 Product Brief. Consultado el 10 de noviembre de 2026 (2026). https://datasheets.raspberrypi.com/rpi5/raspberry-pi-5-product-brief.pdf

[28] Google AI Edge Authors: ai-edge-litert. Paquete de Python en PyPI, versión 2.3.0. Consultado el 10 de noviembre de 2026 (2026). https://pypi.org/project/ai-edge-litert/